Guía del notebook `23_opt_lr_decay`: decaimiento de la tasa de aprendizaje

Puntos: 3 · **Tiempo:** 25 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/23_opt_lr_decay.ipynb`
Requisito: `21` y `22`.

> En el repositorio original del curso este notebook está **vacío**. El de este repositorio es nuevo, hecho con el mismo estilo y la misma función de Beale que el 22.

¿Qué vas a aprender?

- A **reducir la tasa de aprendizaje** mientras se entrena: grande al principio para avanzar rápido, pequeña al final para afinar.
- Tres reglas de decaimiento y cómo programarlas.
- A crear tu propio optimizador **heredando** del de `nni`.

Antes de empezar

1. `conda activate nni`, y `git pull` para tener este notebook.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/23_opt_lr_decay.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

Con una tasa **fija** hay un dilema: si es grande, rebota cerca del mínimo; si es pequeña, tarda muchísimo. Con **decaimiento**, empezamos grande y la vamos bajando.

Regla	Fórmula	En Python
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$	cada `cada` iteraciones multiplica por `gamma`
Inversa	$\eta_0 / (1 + k \cdot t)$	baja rápido al inicio y luego despacio
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$	baja suave y continuo (`math.exp`)

- t es el número de iteración (0, 1, 2…).
- `t // s` es la división entera: 250 // 100 = 2.

Paso a paso

Celdas 1–3: paquetes y función de Beale

Igual que en el notebook 22. El mínimo está en (3, 0.5).

Celda `TODO (1 punto)`: las tres reglas

Escribe las tres funciones con las fórmulas de la tabla.

✅ **Comprobación:** la celda siguiente debe imprimir:

```
t =   0:  escalonada = 0.05000   inversa = 0.05000   exponencial = 0.05000
t = 100:  escalonada = 0.02500   inversa = 0.02500   exponencial = 0.01839
t = 250:  escalonada = 0.01250   inversa = 0.01429   exponencial = 0.00410
```

Celda `TODO (1 punto)`: `DecayOptimizer.next_params`

Sigue los 4 pasos escritos en los comentarios:

1. Calcula la tasa de esta iteración: `self.regla(self.lr0, self.t)`.
2. Guárdala en `self.tasas` (con `.append`) y en `self.learning_rate`.
3. Suma 1 a `self.t`.
4. Devuelve los parámetros nuevos: para cada par (parámetro, gradiente), parámetro − tasa · gradiente. Pista: es igual a la clase del notebook 21, pero con la tasa que calculaste.

Celda: comparar tasa fija y decaimiento

Desde (1.0, 1.0), con $\eta_0 = 0.05$ y 500 iteraciones.

✅ **Comprobación:**

```
Tasa fija 0.05:       termina en (2.866, 0.409),  f = 5.76e-02
Con decaimiento:      termina en (2.980, 0.495),  f = 6.34e-05
```

Con decaimiento, el error es casi **mil veces menor**. La gráfica muestra:

- con tasa fija, el camino **rebotando** dentro del valle;
- con decaimiento, el camino asentándose en el mínimo;
- la tasa de aprendizaje bajando "en escalones".

Celda `TODO (1 punto)`: experimenta

✅ **Resultados de referencia** (desde (1.0, 1.0), 500 iteraciones):

Configuración	Termina en	f
Escalonada (γ = 0.5 cada 100)	(2.980, 0.495)	$6.3 \cdot 10^{-5}$
Inversa (k = 0.01)	(2.975, 0.494)	$1.0 \cdot 10^{-4}$
Exponencial (k = 0.01)	(2.908, 0.476)	$1.5 \cdot 10^{-3}$
Tasa fija 0.01	(2.910, 0.477)	$1.4 \cdot 10^{-3}$
Escalonada con γ = 0.8	**(2.998, 0.499)**	$\mathbf{7.8 \cdot 10^{-7}}$

La exponencial con k = 0.01 **baja demasiado rápido**: en la iteración 250 la tasa ya es 0.004, y se "congela" antes de llegar. Quedó casi igual que la tasa fija pequeña.

Para usar γ = 0.8 sin cambiar la función, puedes pasar una regla "envuelta": `regla=lambda lr0, t: tasa_escalonada(lr0, t, gamma=0.8)`.

Preguntas (orientación)

- **¿Por qué con 0.05 fijo no llega al mínimo?** El paso es demasiado grande para el valle estrecho: salta de una pared a la otra.
- **¿Qué ventaja tiene empezar grande y reducir?** Avanza rápido cuando está lejos y afina cuando está cerca.
- **¿Y si la tasa decae demasiado rápido?** Se queda "congelado" antes de llegar, como la exponencial de la tabla.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... NoneType`	Alguna regla o `next_params` devuelve `None`	Completa todos los `return`
La tasa nunca cambia	No incrementas `self.t`	Agrega `self.t += 1`
La gráfica de la tasa sale vacía	No guardas en `self.tasas`	`self.tasas.append(tasa)`
Resultados distintos al repetir	Reusaste el mismo optimizador (su `t` sigue contando)	Crea uno nuevo en cada prueba